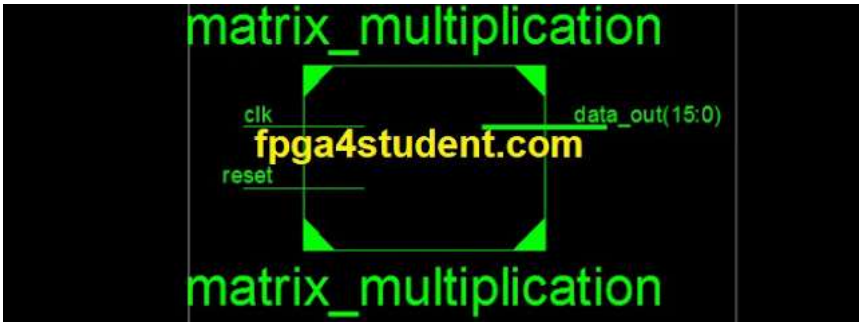
Fixed-Point Matrix Multiplication in Verilog[Full code+Tutorials]

This Verilog project is to implement a synthesizable fixed point matrix multiplication in Verilog HDL. Full Verilog code for the matrix multiplication is presented.

Two fixed point matrixes A and B are BRAMs created by Xilinx Core Generator. After multiplying these two matrixes, the result is written to another matrix which is BRAM. The testbench code reads the content of the output matrix and writes to a "result.dat" file to check the result.



First of all, you need to know what the fixed point means and how it presents in binary numbers. This topic is quite popular and a lot of people already published it, so you can refer to this to get familiar with fixed-point numbers, how it presents in binary numbers, and why we use fixed-point numbers in digital design.

The fixed-point calculations are obviously different from normal binary calculations, so we need a different Verilog library for fixed-point math functions to deal with it on FPGA. Fortunately, we can obtain the Verilog math library for fixed-point numbers from Opencores or you download it directly from here if you don't have an account there. The library contains basic math functions such as addition, multiplication, divisions in Verilog for fixed-point numbers. Thus, what you need to do is downloading the library and spending some time to know the format and how to use the functions for fixed-point calculations in Verilog.

So far, we can deal with fixed-point multiplication for two numbers by using the fixed-point Verilog library. Next, we need to create two BRAMs to store two fixed-point input matrixes. Xilinx Core Generator can help us to create input memories to save two input matrixes. We can use Core Generator to store the initial contents of 2 matrixes for multiplication or we can write input data into the memories in Verilog code. In this project, the first method is used and we will save the contents of two fixed-point matrixes into Matrix_A.coe and Matrix_B.coe, then during synthesis or simulation, these contents are loaded into two input memories. We just need to access these memories and read data out for fixed-point matrix multiplication. Below is an example file for Xilinx .coe :

```
memory_initialization_radix=10;
memory_initialization_vector=
256 256 256 256
256 256 256 256
256 256 256 256
256 256 256 256
```

You can modify them to change the matrix, but it is noted that after modification, regenerate the Core Generator for these cores. Then copy the netlist(Matrix_A.ngc and Matrix_B.ngc) to the folder of ISE project. Below is the code we get from Xilinx Core Generator:

```
LIBRARY ieee;
 USE ieee.std_logic_1164.ALL;
 -- synthesis translate_off
 LIBRARY XilinxCoreLib;
 -- synthesis translate_on
 -- fpga4student.com FPGA projects, Verilog projects, VHDL projects
 -- Verilog project: Verilog code for Fixed-Point Matrix Multiplication
 -- Matrix memory generated by Xilinx Core Generator
 ENTITY Matrix_A IS
```

```
 PORT (
  clka : IN STD_LOGIC;
  addra : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
  douta : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
 );
END Matrix_A;
ARCHITECTURE Matrix_A_a OF Matrix_A IS
-- synthesis translate_off
COMPONENT wrapped_Matrix_A
 PORT (
  clka : IN STD_LOGIC;
  addra : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
  douta : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
 );
END COMPONENT;
-- Configuration specification
 FOR ALL : wrapped_Matrix_A USE ENTITY XilinxCoreLib.blk_mem_gen_v6_1(behavioral)
  GENERIC MAP (
   c_addra_width => 4,
   c_addrb_width => 4,
   c_algorithm => 1,
   c_axi_id_width => 4,
   c_axi_slave_type => 0,
   c_axi_type => 1,
   c_byte_size => 9,
   c_common_clk => 0,
   c_default_data => "0",
   c_disable_warn_bhv_coll => 0,
   c_disable_warn_bhv_range => 0,
   c_family => "spartan6",
   c_has_axi_id => 0,
   c_has_ena => 0,
   c_has_enb => 0,
   c_has_injecterr => 0,
   c_has_mem_output_regs_a => 0,
   c_has_mem_output_regs_b => 0,
   c_has_mux_output_regs_a => 0,
   c_has_mux_output_regs_b => 0,
   c_has_regcea => 0,
   c_has_regceb => 0,
   c_has_rsta => 0,
   c_has_rstb => 0,
   c_has_softecc_input_regs_a => 0,
   c_has_softecc_output_regs_b => 0,
   c_init_file_name => "Matrix_A.mif",
   c_inita_val => "0",
   c_initb_val => "0",
   c_interface_type => 0,
   c_load_init_file => 1,
   c_mem_type => 3,
   c_mux_pipeline_stages => 0,
   c_prim_type => 1,
   c_read_depth_a => 16,
   c_read_depth_b => 16,
   c_read_width_a => 16,
   c_read_width_b => 16,
   c_rst_priority_a => "CE",
   c_rst_priority_b => "CE",
   c_rst_type => "SYNC",
   c_rstram_a => 0,
   c_rstram_b => 0,
   c_sim_collision_check => "ALL",
   c_use_byte_wea => 0,
```

```
    c_use_byte_web => 0,
    c_use_default_data => 0,
    c_use_ecc => 0,
    c_use_softecc => 0,
    c_wea_width => 1,
    c_web_width => 1,
    c_write_depth_a => 16,
    c_write_depth_b => 16,
    c_write_mode_a => "WRITE_FIRST",
    c_write_mode_b => "WRITE_FIRST",
    c_write_width_a => 16,
    c_write_width_b => 16,
    c_xdevicefamily => "spartan6"
   );
-- synthesis translate_on
BEGIN
-- synthesis translate_off
-- fpga4student.com FPGA projects, Verilog projects, VHDL projects
-- Verilog project: Verilog code for Fixed-Point Matrix Multiplication
-- Matrix memory generated by Xilinx Core Generator
U0 : wrapped_Matrix_A
 PORT MAP (
  clka => clka,
  addra => addra,
  douta => douta
 );
-- synthesis translate_on
END Matrix_A_a;

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
-- synthesis translate_off
LIBRARY XilinxCoreLib;
-- synthesis translate_on
 -- fpga4student.com FPGA projects, Verilog projects, VHDL projects
-- Verilog project: Verilog code for Fixed-Point Matrix Multiplication
-- Matrix memory generated by Xilinx Core Generator
ENTITY ROM IS
 PORT (
  clka : IN STD_LOGIC;
  addra : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
  douta : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
 );
END ROM;
ARCHITECTURE ROM_a OF ROM IS
-- synthesis translate_off
COMPONENT wrapped_ROM
 PORT (
  clka : IN STD_LOGIC;
  addra : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
  douta : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
 );
END COMPONENT;
-- Configuration specification
-- fpga4student.com FPGA projects, Verilog projects, VHDL projects
 FOR ALL : wrapped_ROM USE ENTITY XilinxCoreLib.blk_mem_gen_v6_1(behavioral)
  GENERIC MAP (
   c_addra_width => 4,
   c_addrb_width => 4,
   c_algorithm => 1,
   c_axi_id_width => 4,
   c_axi_slave_type => 0,
   c_axi_type => 1,
```

```
    c_byte_size => 9,
    c_common_clk => 0,
    c_default_data => "0",
    c_disable_warn_bhv_coll => 0,
    c_disable_warn_bhv_range => 0,
    c_family => "spartan6",
    c_has_axi_id => 0,
    c_has_ena => 0,
    c_has_enb => 0,
    c_has_injecterr => 0,
    c_has_mem_output_regs_a => 0,
    c_has_mem_output_regs_b => 0,
    c_has_mux_output_regs_a => 0,
    c_has_mux_output_regs_b => 0,
    c_has_regcea => 0,
    c_has_regceb => 0,
    c_has_rsta => 0,
    c_has_rstb => 0,
    c_has_softecc_input_regs_a => 0,
    c_has_softecc_output_regs_b => 0,
    c_init_file_name => "ROM.mif",
    c_inita_val => "0",
    c_initb_val => "0",
    c_interface_type => 0,
    c_load_init_file => 1,
    c_mem_type => 3,
    c_mux_pipeline_stages => 0,
    c_prim_type => 1,
    c_read_depth_a => 16,
    c_read_depth_b => 16,
    c_read_width_a => 16,
    c_read_width_b => 16,
    c_rst_priority_a => "CE",
    c_rst_priority_b => "CE",
    c_rst_type => "SYNC",
    c_rstram_a => 0,
    c_rstram_b => 0,
    c_sim_collision_check => "ALL",
    c_use_byte_wea => 0,
    c_use_byte_web => 0,
    c_use_default_data => 0,
    c_use_ecc => 0,
    c_use_softecc => 0,
    c_wea_width => 1,
    c_web_width => 1,
    c_write_depth_a => 16,
    c_write_depth_b => 16,
    c_write_mode_a => "WRITE_FIRST",
    c_write_mode_b => "WRITE_FIRST",
    c_write_width_a => 16,
    c_write_width_b => 16,
    c_xdevicefamily => "spartan6"
   );
-- synthesis translate_on
BEGIN
-- synthesis translate_off
U0 : wrapped_ROM
 PORT MAP (
  clka => clka,
  addra => addra,
  douta => douta
 );
```

```
 -- synthesis translate_on
 END ROM_a;
```

To save the result of the fixed-point matrix multiplication, we need one more output memory and we can use Core Generator to create it. It is noticed that this memory is different from these two memories because it should have input and output ports to write data into and get data out. Below is the core from Xilinx Core Generator for the output memory:

```
LIBRARY ieee;
 USE ieee.std_logic_1164.ALL;
 -- synthesis translate_off
 LIBRARY XilinxCoreLib;
 -- synthesis translate_on
 -- fpga4student.com FPGA projects, Verilog projects, VHDL projects
 -- Verilog project: Verilog code for Fixed-Point Matrix Multiplication
 -- Matrix memory generated by Xilinx Core Generator for storing matrix multiplica
 ENTITY matrix_out IS
  PORT (
   clka : IN STD_LOGIC;
   wea : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
   addra : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
   dina : IN STD_LOGIC_VECTOR(15 DOWNTO 0);
   douta : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
  );
 END matrix_out;
 ARCHITECTURE matrix_out_a OF matrix_out IS
 -- synthesis translate_off
 COMPONENT wrapped_matrix_out
  PORT (
   clka : IN STD_LOGIC;
   wea : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
   addra : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
   dina : IN STD_LOGIC_VECTOR(15 DOWNTO 0);
   douta : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
  );
 END COMPONENT;
 -- Configuration specification
 -- fpga4student.com FPGA projects, Verilog projects, VHDL projects
 -- Matrix memory generated by Xilinx Core Generator for storing matrix multiplica
  FOR ALL : wrapped_matrix_out USE ENTITY XilinxCoreLib.blk_mem_gen_v6_1(behaviora
   GENERIC MAP (
    c_addra_width => 4,
    c_addrb_width => 4,
    c_algorithm => 1,
    c_axi_id_width => 4,
    c_axi_slave_type => 0,
    c_axi_type => 1,
    c_byte_size => 9,
    c_common_clk => 0,
    c_default_data => "0",
    c_disable_warn_bhv_coll => 0,
    c_disable_warn_bhv_range => 0,
    c_family => "spartan6",
    c_has_axi_id => 0,
    c_has_ena => 0,
    c_has_enb => 0,
    c_has_injecterr => 0,
    c_has_mem_output_regs_a => 0,
    c_has_mem_output_regs_b => 0,
    c_has_mux_output_regs_a => 0,
    c_has_mux_output_regs_b => 0,
    c_has_regcea => 0,
    c_has_regceb => 0,
    c_has_rsta => 0,
```

```
    c_has_rstb => 0,
    c_has_softecc_input_regs_a => 0,
    c_has_softecc_output_regs_b => 0,
    c_init_file_name => "no_coe_file_loaded",
    c_inita_val => "0",
    c_initb_val => "0",
    c_interface_type => 0,
    c_load_init_file => 0,
    c_mem_type => 0,
    c_mux_pipeline_stages => 0,
    c_prim_type => 1,
    c_read_depth_a => 16,
    c_read_depth_b => 16,
    c_read_width_a => 16,
    c_read_width_b => 16,
    c_rst_priority_a => "CE",
    c_rst_priority_b => "CE",
    c_rst_type => "SYNC",
    c_rstram_a => 0,
    c_rstram_b => 0,
    c_sim_collision_check => "ALL",
    c_use_byte_wea => 0,
    c_use_byte_web => 0,
    c_use_default_data => 0,
    c_use_ecc => 0,
    c_use_softecc => 0,
    c_wea_width => 1,
    c_web_width => 1,
    c_write_depth_a => 16,
    c_write_depth_b => 16,
    c_write_mode_a => "WRITE_FIRST",
    c_write_mode_b => "WRITE_FIRST",
    c_write_width_a => 16,
    c_write_width_b => 16,
    c_xdevicefamily => "spartan6"
   );
 -- synthesis translate_on
 BEGIN
 -- synthesis translate_off
 U0 : wrapped_matrix_out
  PORT MAP (
   clka => clka,
   wea => wea,
   addra => addra,
   dina => dina,
   douta => douta
  );
 -- synthesis translate_on
 END matrix_out_a;
```

It can be easily seen that it has input ports to enable writing into the memory and also reading data out. This project is to calculate a fixed point multiplication for 4x4 matrixes. The technique being used for matrix multiplication is mentioned before in the previous post: VHDL code for matrix multiplication. You can refer to this if you are looking for the VHDL version of matrix multiplication.

Below is the Verilog code for fixed-point matrix multiplication:

```
`timescale 1ns / 1ps
 // Fixed point 4x4 Matrix Multiplication
 // fpga4student.com FPGA projects, Verilog projects, VHDL projects
 // Verilog project: Verilog code for fixed point Matrix multiplication
 module matrix_multiplication(
           input clk,reset,
      output [15:0] data_out
   );  // fpga4student.com FPGA projects, Verilog projects, VHDL projects
```

```
       // Input and output format for fixed point
      //     |1|<- N-Q-1 bits ->|<--- Q bits -->|
      // |S|IIIIIIIIIIIIIIII|FFFFFFFFFFFFFFF|
wire [15:0] mat_A;
wire [15:0] mat_B;
wire overflow1,overflow2,overflow3,overflow4;
reg wen;
reg [15:0]data_in;
reg [3:0] addr;
reg [4:0] address;
reg [15:0] matrixA[3:0][3:0],matrixB[3:0][3:0];
//wire [15:0] matrix_output[3:0][3:0];
wire [15:0] tmp1[3:0][3:0],tmp2[3:0][3:0],tmp3[3:0][3:0],tmp4[3:0][3:0],tmp5[3:0]
      // BRAM matrix A
      Matrix_A matrix_A_u (.clka(clk),.addra (addr),.douta(mat_A) );
      // BRAM matrix B
       ROM matrix_B_u(.clka(clk), .addra (addr),.douta(mat_B) );
      always @(posedge clk or posedge reset)
      begin
           if(reset) begin
                addr <= 0;
           end
           else
           begin
                if(addr<15)
                addr <= addr + 1;
                else
                addr <= addr;
                matrixA[addr/4][addr-(addr/4)*4] <= mat_A ;
                matrixB[addr/4][addr-(addr/4)*4] <= mat_B ;
           end
      end
      // fpga4student.com FPGA projects, Verilog projects, VHDL projects
      genvar i,j,k;
      generate
      for(i=0;i<4;i=i+1) begin:gen1
      for(j=0;j<4;j=j+1) begin:gen2
           // fixed point multiplication
           qmult #(8,16) mult_u1(.i_multiplicand(matrixA[i][0]),.i_multiplier(matr
           qmult #(8,16) mult_u2(.i_multiplicand(matrixA[i][1]),.i_multiplier(matr
           qmult #(8,16) mult_u3(.i_multiplicand(matrixA[i][2]),.i_multiplier(matr
           qmult #(8,16) mult_u4(.i_multiplicand(matrixA[i][3]),.i_multiplier(matr
           // fixed point addition
           qadd #(8,16) Add_u1(.a(tmp1[i][j]),.b(tmp2[i][j]),.c(tmp5[i][j]));
           qadd #(8,16) Add_u2(.a(tmp3[i][j]),.b(tmp4[i][j]),.c(tmp6[i][j]));
           qadd #(8,16) Add_u3(.a(tmp5[i][j]),.b(tmp6[i][j]),.c(tmp7[i][j]));
           //assign matrix_output[i][j]= tmp7[i][j];
      end
      end
      endgenerate
      // fpga4student.com FPGA projects, Verilog projects, VHDL projects
      always @(posedge clk or posedge reset)
      begin
           if(reset) begin
                address <= 0;
                wen <= 0;
                end
           else begin
                address <= address + 1;
                if(address<16) begin
                     wen <= 1;
                     data_in <= tmp7[address/4][address-(address/4)*4];
                end
```

```
                else
                begin
                     wen <= 0;
                end
           end
      end
      matrix_out matrix_out_u(.clka(clk),.addra (address[3:0]),.douta(data_out),.w
 endmodule
```

Testbench Verilog code for matrix multiplication:

```
 `timescale 10ns / 1ps
 module tb_top;  // fpga4student.com FPGA projects, Verilog projects, VHDL project
      // Inputs
      reg clk;
      reg reset;
      integer i;
      wire [15:0] data_out;
      reg [15:0] matrix_out[15:0];
      integer fd;
      parameter INFILE = "result.dat";
      // Instantiate the Unit Under Test (UUT)
      matrix_multiplication uut (
           .clk(clk),
           .reset(reset),
           .data_out(data_out)
      );
      initial begin
           // Initialize Inputs
           reset = 1;
           clk <= 0;
           // Wait 100 ns for global reset to finish
           #100;
           reset = 0;
           for(i=0;i<32;i=i+1)
           begin
                #100 clk = ~clk;
           end
           #10000
           reset = 1;
           #1000
           reset = 0;
           for(i=0;i<32;i=i+1)
           begin
                #100 clk = ~clk;
           end
           for(i=0;i<64;i=i+1)
           begin
                #100 clk = ~clk;
           end
           clk = 0;
           for(i=0;i<32;i=i+1)
           begin
                 #100 clk = ~clk;
                 matrix_out[i/2] = data_out;
           end
           #100;
             for(i=0; i<16; i=i+1) begin
                  $fwrite(fd, "%d", matrix_out[i][15:8]);
                  $fwrite(fd, "%d", matrix_out[i][7:0]);
                  #200;
                end
           end
 // fpga4student.com FPGA projects, Verilog projects, VHDL projects
```

```
// Writing the output result to result.dat file
   initial begin
                fd = $fopen(INFILE, "wb+");
          end
endmodule
```

The Verilog code for fixed-point matrix calculation is synthesizable and can be implemented on FPGA. The simulation result is written into the result.dat file and we can easily check the result from the file.

Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**
36. **Verilog code for Moore FSM Sequence Detector**
37. **Verilog code for 7-segment display controller on Basys 3 FPGA**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.